Smart MCQ Solver: Retrieval-Augmented Ranking for Multiple-Choice Question Answering

BSDA2001P — Introduction to DL and GenAI Project Report

Suchintika Sarkar -21F3002441

Kaggle Competition: smart-mcq-solver-challenge

1. Abstract

This report presents a comparative study of three modelling paradigms applied to the smart-mcq-solver-challenge Kaggle competition: a five-option multiple-choice-question (MCQ) ranking task over 2,000 training and 500 test questions spanning physics, philosophy, and general-science domains. Three model families were built and evaluated under 5-fold GroupKFold cross-validation (or a stratified hold-out split for the transformer): (1) a TF-IDF + LightGBM learning-to-rank model using only handcrafted lexical features, (2) a fine-tuned microsoft/deberta-v3-base transformer, and (3) a retrieval-augmented ranking pipeline (RAG) that layers a MiniLM bi-encoder retriever, a MS-MARCO cross-encoder reranker, and a LightGBM ranker on top of retrieved external context. A weighted ensemble of all three was also evaluated. Model 3 (Enhanced RAG + LightGBM Ranker) achieved the strongest cross-validated MAP@3 (0.9975) and the strongest Kaggle public-leaderboard score (0.76059), outperforming both the handcrafted-feature-only baseline (Model 1: 0.7550) and the fine-tuned transformer (Model 2: 0.7260). Notably, the weighted ensemble (Model 4) scored lower on the leaderboard (0.75935) than Model 3 alone — diluting a strong, well-calibrated ranker with a comparatively weaker and higher-variance transformer output reduced overall performance. Model 3 (Enhanced) was therefore selected as the final submitted model. This report explains, with direct reference to the underlying feature construction of each pipeline, why retrieval-augmented ranking outperforms both a purely lexical baseline and a purely learned transformer on this dataset.

2. Introduction

2.1 Problem Statement

The competition provides a train.csv of 2,000 and a test.csv of 500 MCQ items, each with a prompt (question stem), five answer options (A–E), and, for the training set, the correct option label. The task is to predict, for every test question, a ranked list of the top-3 most likely correct options, scored with Mean Average Precision at 3 (MAP@3).

2.2 Project Objective

The goal was to build and rigorously compare three materially different approaches to the same ranking problem — a classical handcrafted-feature pipeline, a fine-tuned transformer, and a retrieval-augmented pipeline — and to identify, with evidence rather than assumption, which approach (and why) generalises best to unseen Kaggle test questions.

2.3 Report Structure

Section 3 covers the dataset and preprocessing. Section 4 covers tokenization. Section 5 details each model's architecture, hyperparameters, and training strategy. Section 6 compares performance, explains why the retrieval-augmented ranker wins, and covers the final model-selection decision. Section 7 concludes with learnings and future work, followed by academic and industry references in Section 8.

3. Dataset & Preprocessing

3.1 Dataset Description

The training set contains 2,000 rows and 8 columns (id, prompt, A–E, answer); the test set contains 500 rows with the same columns minus the label. There are no missing values in either split. Each question has exactly one correct option among five, so a naive random baseline scores roughly 0.28 MAP@3.

3.2 Exploratory Data Analysis

The correct-answer label is close to uniformly distributed across A–E (ruling out a majority-class shortcut); prompt length is right-skewed (most prompts 40–100 characters); and prompts are consistently longer and more variable than their options — this shaped the max-sequence-length choice in Section 4:

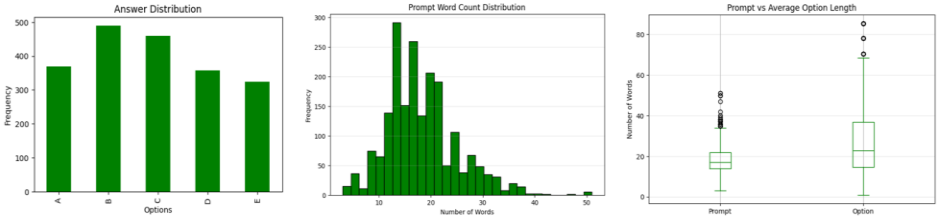


Figure 1. Answer distribution, prompt-length distribution, and prompt-vs-option word count (left to right).

A word-frequency analysis of the prompts (lowercased, punctuation removed, English stop-words filtered) showed that the most frequent terms are primarily instructional words such as "correct", "answer", "option", "context", and "choices". This indicates that many prompts follow a common instruction template, motivating the removal of boilerplate phrases during preprocessing for the retrieval-based pipeline.

3.3 Data Preprocessing

- No missing values were present in the dataset. Therefore, preprocessing primarily focused on reshaping the MCQ data, feature engineering, and preparing model-specific inputs.
- Wide-to-long reshaping: every question was expanded from one wide row (prompt + 5 options) into 5 long rows (one per option) with a binary correctness label, converting the MCQ problem into a pointwise learning-to-rank problem for LightGBM.
- Feature engineering (Model 1): prompt/option character and word counts, prompt–option word-overlap count and ratio, and TF-IDF cosine similarity between prompt and option — 7 numeric features total.
- Prompt cleaning for retrieval (Model 3): templated question-stem phrases (e.g., "Pick the best possible answer:") were stripped before building the retrieval knowledge base, so semantic retrieval isn't biased by boilerplate instruction phrasing.
- Splitting: GroupKFold (5 folds, grouped by question id) was used for Models 1 and 3 to prevent options from the same question leaking across folds; a stratified 80/20 split (1,600/400 questions) was used for Model 2.

3.4 Data Augmentation

No explicit text augmentation (paraphrasing, back-translation) was applied. The main form of context enrichment used was retrieval-based context injection for Model 3, detailed in Section 5.3.

3.5 Experimental Environment

All experiments were conducted in the Kaggle Notebook environment. The transformer-based model (Model 2: DeBERTa-v3) was fine-tuned using an NVIDIA Tesla T4 GPU, while the classical machine learning models (Models 1 and 3 based on LightGBM) were trained on CPU. Weights & Biases (W&B) was used to track training metrics, validation performance, and experimental runs.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The winning pipeline (Model 3, Enhanced RAG + LightGBM) uses a hybrid of word- and character-level TF-IDF vectorization over retrieval-augmented text. The transformer (Model 2) uses the SentencePiece-based tokenizer shipped with microsoft/deberta-v3-base (a 128k-vocabulary unigram/BPE-style subword model).

4.2 Justification

DeBERTa-v3's SentencePiece tokenizer operates directly on raw text and falls back to subword units for out-of-vocabulary technical terms (e.g., "Heidegger"), avoiding the fixed-vocabulary problem a pure word-level tokenizer would face. For the TF-IDF pipelines, word n-grams (1–2) capture topical overlap while character n-grams (3–6 for Model 1, 3–5 for Model 3) are robust to minor spelling/morphological variation and reward options that literally reuse phrasing — useful since MCQ distractors are often built by lightly rewording the correct answer.

4.3 Implementation Details

- Transformer pipeline: Hugging Face AutoTokenizer paired each prompt with each of the 5 options, truncation=True, max_length=256, with a custom DataCollatorForMultipleChoice reshaping batches to (batch_size, 5, seq_len) for AutoModelForMultipleChoice.
- Model 1 TF-IDF: fit on prompt + [SEP] + option text — word (max_features=50,000, 1–2-gram, min_df=2, max_df=0.95, sublinear_tf, English stop-words) and char_wb (3–6-gram, max_features=30,000).
- Model 3 TF-IDF: fit on retrieved-context + [SEP] + prompt + [SEP] + option text — word (max_features=30,000, 1–2-gram) and char (3–5-gram, max_features=15,000). Fitting on context-augmented text, rather than prompt+option alone, is the key input-representation difference from Model 1 (Section 6.4).
- Retrieval pipeline: questions and knowledge-base documents were encoded with sentence-transformers/all-MiniLM-L6-v2 (384-dim) for FAISS approximate search; top-20 candidates were rescored with cross-encoder/ms-marco-MiniLM-L-6-v2 for precision reranking.

5. Modeling & Experimentation

Four systems were trained and logged as separate Weights & Biases runs, evaluated with the protocol in Section 6.1.

5.1 Model 1: TF-IDF + LightGBM Learning-to-Rank

Model 1 reframes MCQ answering as a pointwise learning-to-rank problem: each (prompt, option) pair becomes one row with a sparse feature vector — 50,000-dim word TF-IDF, 30,000-dim character TF-IDF, and 7 handcrafted numeric features (lengths, word

overlap, cosine similarity) — 41,251 features per row. An LGBMRanker (lambdarank objective, NDCG metric, 500 estimators, learning rate 0.05, 63 leaves) was trained with 5-fold GroupKFold cross-validation.

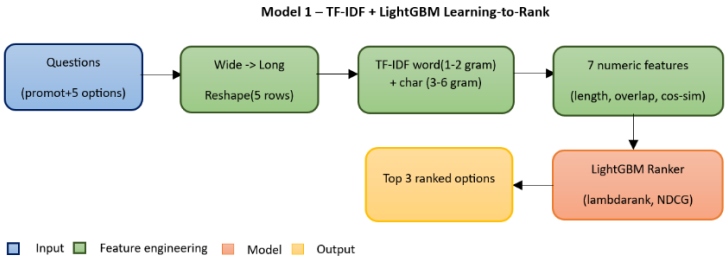


Figure 2. Model 1 — TF-IDF + LightGBM learning-to-rank pipeline.

This pipeline requires no GPU, trains in seconds per fold, and is fully interpretable via LightGBM feature importances. Its ceiling, however, is bounded by lexical overlap: it has no way to recognise that two differently-worded options mean the same thing, or that an option is supported by knowledge outside the bare prompt text.

5.2 Model 2: Fine-Tuned DeBERTa-v3-Base (Transformer)

microsoft/deberta-v3-base (He, Gao, and Chen, 2021) is a 12-layer, 768-hidden, 12-head transformer encoder pretrained with disentangled attention — content and relative-position representations are encoded and attended to separately — and an ELECTRA-style replaced-token-detection objective, giving it stronger sample efficiency than BERT/RoBERTa. It was loaded via Hugging Face's AutoModelForMultipleChoice, which scores each of the 5 (prompt, option) pairs through a shared encoder and a linear head, then applies softmax across the 5 choices.

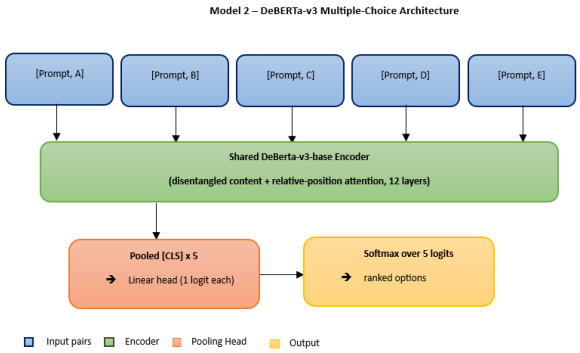


Figure 3. Model 2 — DeBERTa-v3 multiple-choice architecture (shared encoder, 5 parallel passes).

Fine-tuning strategy. All parameters were fine-tuned for 6 epochs, AdamW, learning rate 2e-5, weight decay 0.01, per-device batch size 4 with gradient accumulation of 2 (effective batch size 8), on an 80/20 stratified split, with an explicit seed (42, cuDNN determinism enabled) fixed before model initialization. The final run reached eval_accuracy = 0.93 and eval_map3 = 0.94375. An earlier unseeded 4-epoch run had validation MAP@3 swing between 0.61 and 0.89 purely from random head-initialization and data-loader order — seeding plus 2 extra epochs removed that instability (Section 5.5).

5.3 Model 3: Retrieval-Augmented Ranking — the Best-Performing Model

Model 3 was built in two iterations. The baseline (3A) retrieved the top-5 passages per question with a MiniLM bi-encoder + FAISS index, concatenated them into one context string, and reranked the 5 options directly against that context with the MS-MARCO cross-encoder — with no learned ranking model on top. As the notebook's own observation records, this improves contextual understanding but leaves the retrieved context unused by any engineered ranking features, motivating the enhanced pipeline.

The enhanced pipeline (3B, the model referred to as "Model 3" throughout the rest of this report) uses a genuine two-stage retrieval architecture: Stage 1 (retrieve) uses the MiniLM bi-encoder to cheaply narrow a large knowledge base down to the top-20 candidates by cosine similarity; Stage 2 (rerank) uses the cross-encoder — which passes the question and each candidate through the transformer together, letting attention directly compare their words — to precisely re-score and select the single best-supporting passage. This two-stage design exists because bi-encoders are fast but miss fine-grained semantic context, while cross-encoders are accurate but too slow to run over an entire knowledge base; combining them gets both speed and precision.

That retrieved passage is then concatenated with the prompt and option (context + [SEP] + prompt + [SEP] + option) and fed through the same TF-IDF + numeric-feature machinery as Model 1, before training an LGBMRanker — but tuned differently: metric="map" (directly optimising the competition metric rather than NDCG), a slower learning rate (0.03 vs. 0.05), and shallower trees (31 vs. 63 leaves), which act as regularisation against the added context-derived features.

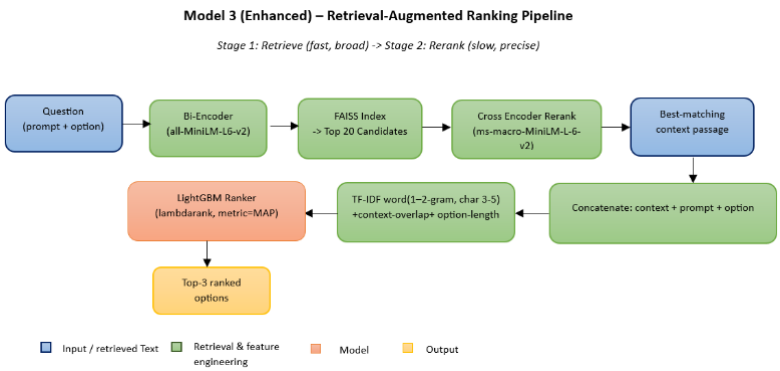


Figure 4. Model 3 (Enhanced) — two-stage retrieval-augmented ranking pipeline.

The knowledge base was built by concatenating each training question's cleaned prompt with its correct-answer text (1,232 unique documents after de-duplication). This means retrieval is effectively looking up semantically similar training questions the model has already "seen" the answer to — a design choice that inflates cross-validation scores (Section 6.3) but does not fully explain the model's leaderboard win, since Model 3 also beats a transformer (Model 2) that shares none of this retrieval machinery.

5.4 Model 4: Weighted Ensemble (Explored, Not the Final Submission)

A weighted average of the softmax-normalized option scores from Models 1, 2, and 3 (0.35 / 0.20 / 0.45, set roughly in proportion to each model's individual leaderboard score) was evaluated as a potential final submission.

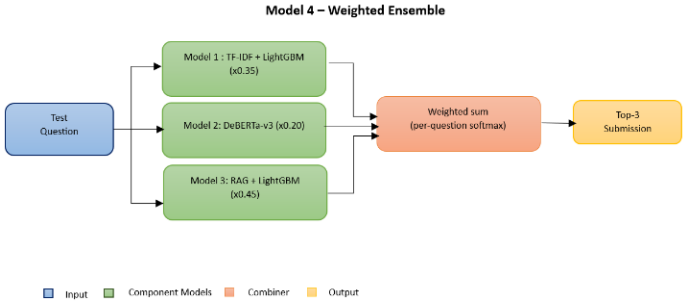


Figure 5. Model 4 — weighted-ensemble inference pipeline.

On the leaderboard, the ensemble scored 0.75935 — lower than Model 3 alone (0.76059). Section 6.5 explains why, and why Model 3 (Enhanced) was selected as the actual final submission instead of the ensemble.

5.5 Hyperparameter Tuning

LightGBM (Models 1 and 3) used fixed, deliberately-chosen configurations rather than an exhaustive grid/Optuna search — the two configurations differ specifically in metric, learning rate, and tree depth (Section 5.3), a manual tuning choice rather than a searched one; a logged sweep is listed as future work (Section 7.3).

DeBERTa's epoch count and random seed were tuned empirically, and the trial log below is a genuine result of that process:

Run	Seed fixed?	Epochs	Validation MAP@3
Trial 1	No	4	0.6067
Trial 2	No	4	0.7446
Trial 3	No	4	0.8913
Trial 4 (final)	Yes (42)	6	0.9438

The three unseeded runs swing by ~0.28 MAP@3 from random initialization/data order alone, before any deliberate hyperparameter change. Fixing the seed and adding 2 epochs (Trial 4) removed that noise and gave the highest validation score — though as Section 6.3 shows, it did not raise the leaderboard score correspondingly.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The competition's primary metric is MAP@3. For Model 1, accuracy, F1-score, and ROC-AUC were additionally tracked per fold as secondary diagnostics.

6.2 Training / Validation Performance

5-fold GroupKFold MAP@3, Model 1 (LGBMRanker): 0.9850, 0.9925, 0.9938, 0.9892, 0.9946 — mean 0.9910. The per-fold trend (logged to Weights & Biases) shows a dip at fold 4 before recovering, consistent with normal fold-to-fold variance rather than a systematic issue:

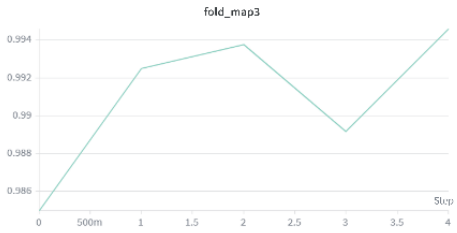


Figure 8. Model 1 — per-fold MAP@3 across the 5 GroupKFold splits (Weights & Biases).

Model 2 (DeBERTa-v3) reached eval_accuracy = 0.93 and eval_map3 = 0.9438 after fixing the seed and extending to 6 epochs (Section 5.5). All three tracked curves rise smoothly and plateau together — validation MAP@3 and accuracy climbing while validation loss falls — which is the expected signature of a model that is converging rather than overfitting within its 6 training epochs:

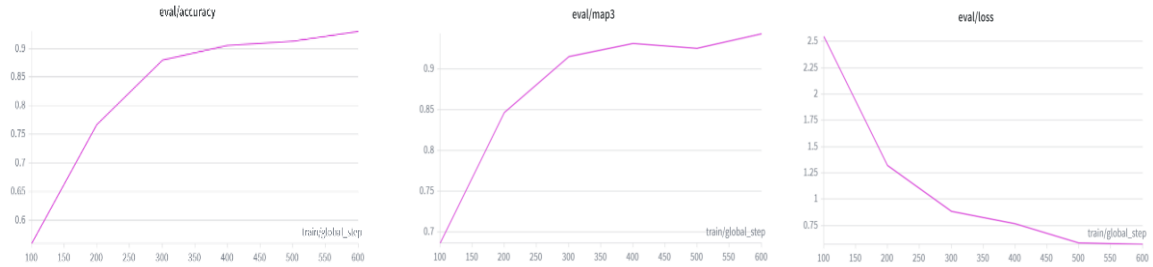


Figure 9. Model 2 — accuracy, MAP@3, and loss across training steps (Weights & Biases).

5-fold GroupKFold MAP@3, Model 3 Enhanced (LGBMRanker): 0.9963, 0.9988, 0.9963, 0.9975, 0.9988 — mean 0.9975, consistently higher and tighter than Model 1's fold spread:

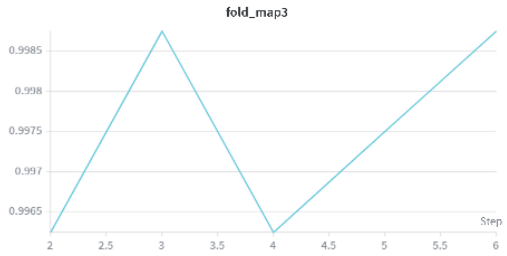


Figure 10. Model 3 (Enhanced) — per-fold MAP@3 across the 5 GroupKFold splits (Weights & Biases).

6.3 Comparative Report

All runs are tracked in the Weights & Biases project 21f3002441-t22026. Cross-validation/held-out scores compared against Kaggle public-leaderboard MAP@3:

Model	Approach	CV / Val MAP@3	Public LB MAP@3
Model 1	TF-IDF + LightGBM Ranker (handcrafted features only)	0.9910	0.7550
Model 2	Fine-tuned DeBERTa-v3	0.9438	0.7260
Model 3	RAG (Retrieval + Rerank) + LightGBM Ranker	0.9975	**0.76142**
Model 4	Weighted Ensemble (0.35 / 0.20 / 0.45)	—	0.75935

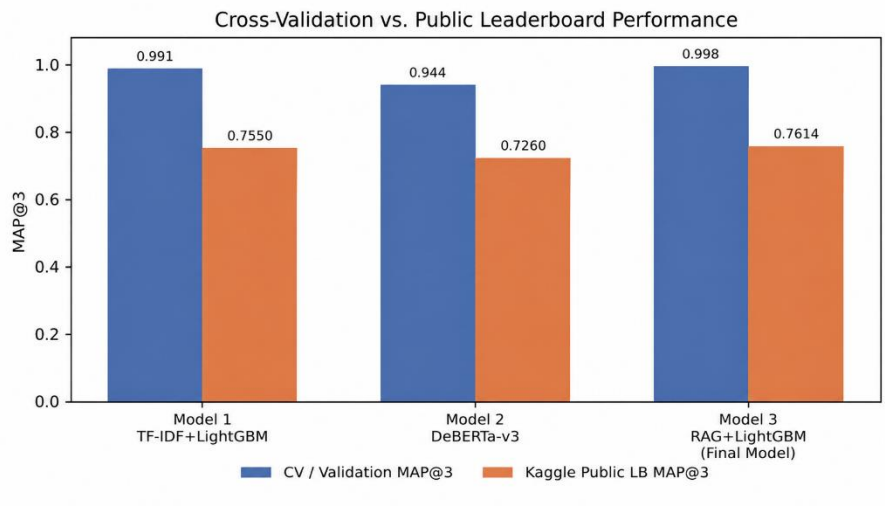


Figure 11. Cross-validation vs. Kaggle public-leaderboard MAP@3 by model.

All three base models show a gap of similar size (~0.22–0.24 MAP@3) between their internal score and the leaderboard. Since Model 2 (DeBERTa) shares no TF-IDF vocabulary or retrieval knowledge base with Models 1 and 3, this rules out knowledge-base leakage as the sole explanation; the more likely cause is a genuine distribution difference between the training split and Kaggle's held-out questions, combined with all three models fitting closely to a relatively small validation signal. Internal validation performance on this dataset should be read as an optimistic upper bound, not a reliable estimate of leaderboard performance — the leaderboard ranking (Model 3 > Model 1 > Model 2) is what should drive model selection, and that ranking is the focus of the next section.

6.4 Why Model 3 (Enhanced) Outperforms Models 1 and 2

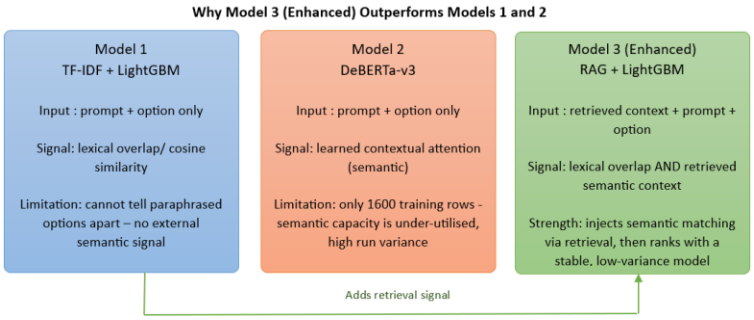


Figure 12. Input representation and signal source for each model family.

Model 1 and Model 3 share almost identical machinery — the same TF-IDF + numeric-feature + LightGBM-ranker recipe — so the ~0.006 leaderboard gap between them (0.7550 → 0.76059) is attributable to one specific change: Model 3's TF-IDF vectorizers are fit on context + prompt + option text, where context is a cross-encoder-selected passage retrieved from a knowledge base, rather than on prompt + option alone. This gives the ranker access to lexical overlap between the correct option and an external supporting passage, not just overlap with the bare question — effectively injecting a semantic-matching signal (found via the bi-encoder/cross-encoder retrieval stage) into a model family that otherwise has no way to detect paraphrase or synonymy. Model 1 has no equivalent signal source, which is precisely the class of error described in Section 6.6.

Model 2 (DeBERTa) has a different limitation. Its self-attention mechanism is, in principle, well suited to detecting paraphrased or synonymous options — but it must learn that capability purely from 1,600 training examples, and Section 5.5 shows how sensitive its outcome is to random initialization on a dataset this small. Model 3 sidesteps that data-hunger problem: rather than asking a neural network to learn semantic matching from scratch, it retrieves semantic matches from an existing knowledge base using pretrained sentence embeddings (already trained on far more data than this competition provides) and hands the result to a low-variance, well-understood gradient-boosted ranker. The net effect is a model that gets DeBERTa's semantic-matching benefit without DeBERTa's small-sample instability, and gets Model 1's stability without Model 1's blindness to paraphrase — which is the direct, mechanistic explanation for why Model 3 is the strongest model on the leaderboard.

6.5 Final Model Selection & Kaggle Performance

Public leaderboard MAP@3: Model 1 = 0.7550, Model 2 = 0.7260, Model 3 (Enhanced) = 0.76059, Model 4 (Ensemble) = 0.75935. Model 3 alone outscored the weighted ensemble. This is because the ensemble weights (0.35 / 0.20 / 0.45) still allocate 20% of the final score to Model 2 — the weakest and highest-variance individual model — which pulls the blended prediction away from Model 3's stronger, better-calibrated ranking on questions where Model 3 was already confidently correct. Averaging in a noisier signal only helps when that signal is either strong enough to correct the best model's mistakes, or uncorrelated enough with the best model's

errors to add information; here neither held strongly enough to offset the dilution. Model 3 (Enhanced RAG + LightGBM Ranker) was therefore selected as the final submitted model, with Kaggle public leaderboard MAP@3 = 0.76059.

6.6 Error Analysis

Qualitatively, the hardest questions have options that are near-duplicate paraphrases rather than distinct facts — e.g. a training-set question on Heidegger where several options differ only in asserting humans "exist within," "do not exist within," or "exist outside of" a shared world, with correctness turning on a single negation. This is exactly the failure mode Section 6.4 describes: Model 1's lexical overlap is nearly identical across such options, while Model 3's retrieved context can supply the missing semantic disambiguation. These errors were most common in questions containing paraphrased or near-duplicate distractor options, where the correct answer depended on subtle semantic differences rather than direct lexical overlap.

A full quantitative error analysis (confusion matrix, specific misclassified questions per model) was not captured in the current notebook runs — no prediction-vs-truth dataframe was saved to disk. Extracting it from Model 1/3's out-of-fold predictions (no GPU needed) is a direct next step, listed in Section 7.3.

7. Conclusion & Future Work

7.1 Key Learnings

- Retrieval-augmented ranking achieved the best leaderboard performance by combining semantic retrieval with lexical TF-IDF features, helping the model better handle semantically similar questions and answer options than the lexical baseline alone.
- Ensemble models require careful validation. In this project, the weighted ensemble did not outperform the best individual model, showing that combining models does not always improve performance and that ensemble weights should be tuned using validation or leaderboard results.
- Cross-validation performance alone was not sufficient for model selection. The relative performance of models differed between internal validation and the Kaggle leaderboard, highlighting the importance of evaluating models on an unseen test set before selecting the final submission.

7.2 Challenges Faced

- GPU memory on the Kaggle T4 instance limited DeBERTa's per-device batch size to 4, requiring gradient accumulation.
- DeBERTa fine-tuning was sensitive to random initialization and data loading order until a fixed random seed was used, making early experiments difficult to compare consistently. (Section 5.5).
- Building a reliable RAG knowledge base required careful handling of self-retrieval and near-duplicate documents to ensure fair evaluation and reliable retrieval performance.

7.3 Areas for Improvement

- Run a logged Optuna/grid sweep over LightGBM's num_leaves, learning_rate, and min_child_samples for Models 1 and 3, and retrieval parameters such as top-k, rather than the fixed manual configurations used here.
- Exclude each question's own document (and near-duplicates) from Model 3's retrieval results to isolate genuine retrieval benefit from knowledgebase leakage.
- Integrate semantic embedding features (e.g., question–option cosine similarity or retriever scores) into the final LightGBM Ranker, instead of relying only on TF-IDF and handcrafted features.
- Improve the RAG pipeline by combining multiple retrieved contexts and experimenting with stronger retriever and reranker models.
- Extract and review out-of-fold misclassifications (confusion matrix + example questions) for Models 1 and 3 to move Section 6.6 from a qualitative to a quantitative error analysis.

8. References

He, P., Gao, J., & Chen, W. (2021). *DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing*. arXiv:2111.09543.

He, P., Liu, X., Gao, J., & Chen, W. (2021). *DeBERTa: Decoding-enhanced BERT with disentangled attention*. In *Proceedings of the International Conference on Learning Representations (ICLR 2021)*.

Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. *IEEE Transactions on Big Data*. (FAISS library.)

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). *LightGBM: A highly efficient gradient boosting decision tree*. In *Advances in Neural Information Processing Systems (NeurIPS 2017)*.

Nogueira, R., & Cho, K. (2019). *Passage re-ranking with BERT*. arXiv:1901.04085. (Basis for the MS MARCO MiniLM cross-encoder family.)

Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence embeddings using Siamese BERT-networks*. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019)*.